

Week 2

변수

```
X <- 3      #X
X = 3      #X에
```

price.a 같은 이름으로도 변수를 쓸 수 있음.

벡터 Vector

```
Z=c(1,2,3,4)
Z=c("TOM", "EMMA", "JACK")
Z1=c(1:15)      1부터 1까지
```

seq

```
Z2=seq(from=1, to=15)      #1부터 15까지
Z3=seq(from=1, to=15, by=2)  #1부터 15까지 점프2
Z4=seq(from=1, to=15, length=3) #1부터 15까지를 3등분해라!
```

rep

```
A=rep(1,6)      1을 6번 반복해라
A=rep(1:3,6)     1에서 3까지를 6번 반복해라
A=rep(1:3, each=6) 1에서 3까지를 각각 6번 반복해라
```

형변환

```
as.numeric(x)    x를 숫자형으로
as.logical(x)     x를 논리형으로
as.character(x)   x를 문자형으로
as.factor(x)      x를 팩터형으로
```

팩터 Factor

카테고리형 변수. ex) months = 1, 2, 3, 4, ... , 12
미리 정의된 카테고리만 담길 수 있음.

벡터 연산

```
A=c(1,2,3)      B=c(4,5,6)
A+B
A * B           #요소들끼리 곱하기 c(4,10,18)
B^2             #각 요소들 제곱하기 c(16, 25, 36)
sqrt(B)         제곱
mean(B)         평균
sd(B)           표준편차
log(B)          상용로그 씌운 것
```

벡터 인덱싱

```
A=c(1,2,3,4)라 할
A[1]
```

A[1:3] 1에서 3까지 요소
A[c(1,3)] 1, 3번 요소
A[-2] 2번요소 빼고 나머지

+참고+

A=c("YES","NO","Yes","No","NO") <<이것은 벡터
B=factor(c("Yes","No","Yes","No","No")) << 이것은 팩터 메모리 공간 절약 가능

Week 3

행렬 Matrix

다차원 데이터타입

A=matrix(data=1:6, nrow=3, ncol=2, **byrow=FALSE**)

>>1부터 6까지의 데이터를 행 3개 열2개로 만드는데, 열(행이 FALSE이므로)부터 채워라

B=matrix(data=1:6, nrow=3, ncol=2, **byrow=TRUE**) # 행 순서로 채워라

행렬 합치기

x=c(1,2,3) y=10:12

cbind(x,y) #x와 y를 column방향으로 이어붙인다.

rbind(x,y) #x와 y를 row방향으로 이어붙인다.

matrix(1:6, 3, 2)
== matrix (1:6, nrow=3)

행렬에 이름붙이기

rownames(A) = c("JAN","FEB", "MAR")

colnames(A) = c("price","prom")

데이터프레임 Data Frame

E = data.frame(ID=1:3, AGE=c(20,21,45))

모든 데이터프레임은 열의 길이가 같아야 함
입력할 때 모든 벡터가 길이가 같아야 함.
같은 열끼리는 같은 자료형이어야 함.

summary(E) : 요약

str(E) : 더 요약

행렬 인덱싱

B=matrix(1:6, 3, 2)

B[2,1] #2행 1열에 있는거 주세요

B[1,] #1행 다 주세요

B[c(1,2),2]) #1,2행 중 2열 주세요

B[,2] #2열 다 주세요

B[1:2, 2] #....

데이터프레임 인데싱

```
E = data.frame(ID=1:3, AGE=c(20,21,45))
E$ID           #ID 다주세요
E$ID[2]        #ID에 두 번째 들어있는거 주세요
E[c("ID","AGE")] #ID랑 AGE 주세요
E[c("ID","AGE")][1:2] #ID랑 AGE에 해당하는거 1~2행 주세요
```

dplyr 패키지

#파일 읽기

```
sales = read.csv("cereal sales data.csv")
price = read.csv("cereal price data.csv")
summary(sales) 요약 :
```

JOIN 조인

left_join(a, b, by="x1")	a는 모두 뽑고 a와 b 중 x1이 같은 것이 있으면 데이터 합치기. 빈칸엔 NULL
right_join(a, b, by="x1")	b는 모두 뽑고 a와 b 중 x1이 같은 것이 있으면 데이터 합치기. 빈칸엔 NULL
inner_join(a, b, by="x1")	a, b가 겹치는 행의 데이터만 합치기
full_join(a, b, by="x1")	a, b 둘 중 하나만 있어도 합치기. 빈칸엔 NULL

서브셋 데이터

: 데이터에서 일부만 추출

```
cereal[,2:4]      행은 모두, 열은 2~4열만 추출
cereal[1:10,2:4]  1~10행까지 2~4열만 추출
```

```
cereal[cereal$WEEK <= 20,]  cereal의 WEEK가 20보다 같거나 작은 행, 열은 모두 출력
*참고* 논리연산자 : == != >= <= &(AND) |(OR)
```

subset() 함수 : 행을 골라냄.

2가지 이상 조건으로 subset할 때

두 번째 인자에 &나 |가 있어야 함

```
subset(cereal, WEEK>10 & sales_Wheaties>30)    #WEEK가 10보다 크고, sales_Wheaties가 30보다 큰 것
```

```
subset(cereal, sales_Wheaties>30 | sales_Honey>30)
```

select : 열을 고를 수 있음

```
subset(cereal, select=c(price,Wheaties, sales_Wheaties))
```

filter() 함수 : 행 row을 골라냄. by dplyr

```
filter(cereal, WEEK<10)
```

select() 함수 : 열 col을 골라냄 by dplyr

```
select(cereal, price_Wheaties, sales_Wheaties) << subset 안의 select와 달리 벡터로 묶지않고 나열!
```

랜덤으로 데이터 추출 by dplyr

sample_frac()

```
sample_frac(cereal, size=0.3, replace=FALSE)    #전체 데이터의 30프로를 뽑아줘
```

sample_n()

```
sample_n(cereal, 30, replace=FALSE)             #전체 데이터 중 30개를 뽑아줘
```

```
>> replace가 TRUE이면 중복샘플링 허용, FALSE면 허용 안함
```

파이프 %>%

A %>% B %>% C

전 식의 return 값을 다음 함수의 첫 번째 argument로 전달함.

ex) HP 상품 중 반품인 것을 찾고 싶어요!

```
PC %>% filter(SUB_CATEGORY_DESCRIPTION=="DESKTOP COMPUTERS") %>%  
  #S_C_D가 DESKTOP COMPUTER인 행들만 추출  
select(SUB_CATEGORY_DESCRIPTION, UNIT_PRICE, TRANSACTION_TYPE_DESCRIPTION) %>%  
  #이들 중 열을 SUB_, UNIT_, TRNAS 3개만 남김  
filter(TRANSACTION_TYPE_DESCRIPTION=="HP") %>%  
  #이 중 TRANS 가 "HP"인것만 추출  
filter(UNIT_PRICE<0)  
  #UNIT_PRICE 가 양수
```

%in% 안에 이게 있는가?

vector를 만들고, 이 중 하나인가?

```
set= c('HP','CPQ','PRO')  
PC %>% filter(TRANSACTION_TYPE_DESCRIPTION %in% set)
```

distinct() unique한 값 추출 가능

distinct(열이름) 열 이름이 어떤 게 있는지 알고 싶을 때 사용 가능

```
PC %>% distinct(TRANSACTION_TYPE_DESCRIPTION)  
PC %>% distinct(SUB_CATEGORY_DESCRIPTION, TRANSACTION_TYPE_DESCRIPTION)
```

arrange() 특정 열을 기준으로 정렬

arrange(열이름) 정렬할 때 사용

```
PC %>% distinct(SUB_CATEGORY_DESCRIPTION, TRANSACTION_TYPE_DESCRIPTION) %>%  
  arrange(SUB_CATEGORY_DESCRIPTION)  
PC %>% distinct(SUB_CATEGORY_DESCRIPTION, TRANSACTION_TYPE_DESCRIPTION) %>%  
  arrange(SUB_CATEGORY_DESCRIPTION, TRANSACTION_TYPE_DESCRIPTION)
```

select[-COL_NAME] 특정 열만 빼고 select

```
PC %>% select(-ORIGINAL_TICKET_NBR,-PRODUCT_ID) #이거 빼고 select 해줘
```

데이터프레임에 새로운 열 추가하기

방법 1 : \$을 사용해서 추가하기

```
ms_wheaties = (cereal$sales_Wheaties*cereal$price_Wheaties)/(cereal$sales_Wheaties*ce....)  
cereal$ms_wheaties <- ms_wheaties
```

방법 2 : **mutate()** 사용하기 in dplyr

```
mutate(df, 새로 추가할 이름 = 값을 내용(벡터) )  
cereal <- mutate(cereal, log_price_Wheaties=log(price_Wheaties))
```

방법 2+@ : pipe와 같이 mutate() 사용

```
PC %>% filter(UNIT_PRICE>0) %>%  
  mutate(PRICE_CUT = UNIT_PRICE - EXTENDED_PRICE)
```

rename() 열 이름 바꾸기

rename(새 이름=기존 이름)

1. PC\$brand = PC\$TRANSACTION_TYPE_DESCRIPTION
2. PC %>% rename(brand = TRANSACTION_TYPE_DESCRIPTION)

ifelse() 조건문

ifelse(조건, 참일 때, 거짓일 때)

```
e = c(1,-1,0,2,3,-2)
```

```
ifelse(e>0, "POSITIVE","NEGATIVE")
```

```
PC %>% mutate(category=ifelse(SUB_CATEGORY_DESCRIPTION == "DESKTOP COMPUTERS" |  
SUB_CATEGORY_DESCRIPTION == "NOTEBOOK COMPUTERS",  
"COMPUTER","OTHERS"))
```

case_when() 조건문

case_when(조건1 ~ "aaa", 조건2 ~ "bbb", 조건3 ~ "ccc")

```
PC %>% mutate(price_category = case_when(UNIT_PRICE<500 ~ "low",  
UNIT_PRICE>=500 & UNIT_PRICE<1000 ~ "mid",  
UNIT_PRICE>1000 ~ "high"))
```

pull() 한 열만 vector로

df에서 한 열만 추출해서 vector로 변환

```
PC %>% filter(SUB_CATEGORY_DESCRIPTION == "DESKTOP COMPUTERS") %>%  
pull(UNIT_PRICE) %>% mean()
```

** mean() sd() 같은 함수들은 **vector**만 받을 수 있기 때문에 pull()로 바꿔줘야 함!

summarise() <-> summary랑 다름!!!

df를 그룹별로 나누어 평균/표준편차/최소/최대 등으로 출력할 수 있음

summarise(추가할 변수이름=mean/sd/min/max(행이름), 추가할 변수이름 = sd(행이름))

```
PC %>% filter(SUB_CATEGORY_DESCRIPTION == "DESKTOP COMPUTERS") %>%  
summarise(avg_price=mean(UNIT_PRICE), sd_PRICE=sd(UNIT_PRICE))
```

group_by()

특정 변수를 기준으로 그룹화해 반환함.

group(기준으로 할 열)

말그대로 괄호안에 넣은걸 기준으로 묶음(그룹화)! groupby를 하고 summarise를 하면 나누어진 그룹을 바탕으로 결과가 나옴!

```
PC %>% filter(SUB_CATEGORY_DESCRIPTION=="DESKTOP COMPUTERS") %>%  
group_by(mm) %>% #그룹바이를 하면 평균 내기 전에 "그룹으로 나누어진 걸 바탕으로" 평균을 낸다.  
summarise(n_transactions=n(),avg_price=mean(UNIT_PRICE),sales=sum(UNIT_PRICE*QUANTITY))
```

n() 관측치가 몇 개인지 세는 함수. 파라미터 넣지 않음!!

is.na()

값이 비었나? TRUE이면 비었음. FALSE이면 값이 있음(안비었음)

is.na(열이름)

filter(!is.na(열이름)) #열이름에 값이 들어있으면 출력해라 >> 빈 값은 출력하지 말아라.